

Student Assignment: Customer Churn Prediction & Streamlit Deployment

End-to-End Machine Learning Project

Files provided to students

You are given only:

- customer_churn_data.csv
- This instruction file

You are NOT given a solved notebook or trained model.

Project Goal

Build an end-to-end Machine Learning classification project that predicts whether a customer will churn (Yes or No), save the final trained model/pipeline as a .pkl file, create a Streamlit web app, upload the project to GitHub, and deploy it on Streamlit Community Cloud.

Dataset Columns

The dataset contains these columns:

- customer_id – customer identifier
- age – customer age
- gender – customer gender
- region – geographic region
- tenure_months – number of months the customer has stayed
- monthly_charges – monthly bill amount
- total_charges – total amount charged
- contract_type – Month-to-month / One year / Two year
- internet_service – DSL / Fiber optic / No
- tech_support – technical support status
- online_security – online security status
- paperless_billing – Yes / No
- payment_method – payment method
- num_support_calls – number of support calls
- late_payments_last_year – number of late payments in the last year
- avg_monthly_usage_gb – average monthly internet usage
- churn – TARGET column (Yes / No)

Part A — Jupyter Notebook

Create a notebook named:

model_training.ipynb

Complete ALL of the following inside the notebook.

1. Import Libraries

Import the libraries required for:

- data handling
- visualization
- preprocessing
- model training
- model evaluation
- model saving

2. Load and Understand the Data

Load `customer_churn_data.csv` and show:

- first 5 rows
- last 5 rows
- number of rows and columns
- column names
- data types
- dataset information
- descriptive statistics

3. Data Quality Checks

Find and report:

- missing values in every column
- duplicate records
- unique values in categorical columns
- class distribution of the churn target

4. Data Cleaning

Perform suitable cleaning based on your findings.

You must decide:

- what to do with duplicate records
- how to handle missing numeric values
- how to handle missing categorical values
- whether `customer_id` should be used for model training or removed

Explain your decisions in Markdown cells.

5. Exploratory Data Analysis

Create meaningful visualizations. At minimum include:

- churn class distribution
- churn by contract type
- churn by internet service
- monthly charges distribution
- tenure distribution
- at least ONE additional visualization of your choice

Write 1–2 observations under each important visualization.

6. Prepare Features and Target

Create:

- `X` = input features

- $y = \text{churn}$

Identify:

- numeric columns
- categorical columns

7. Train-Test Split

Split the cleaned data into training and testing sets.

Use a reasonable test size and a fixed `random_state`.

8. Preprocessing

Create suitable preprocessing for mixed data.

Your preprocessing should be able to handle:

- missing numeric values
- missing categorical values
- categorical encoding
- numeric scaling where appropriate

A Scikit-Learn Pipeline / ColumnTransformer is recommended.

9. Train At Least TWO Models

Train and compare at least two classification algorithms.

Examples:

- Logistic Regression
- Random Forest
- Decision Tree
- KNN
- Support Vector Machine

10. Model Evaluation

For EACH model calculate:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix
- Classification Report

Create a comparison table of the models.

11. Select the Final Model

Choose the better model based on suitable evaluation metrics.

Do NOT choose only by looking at training accuracy.

Explain why you selected it.

12. Save the Final Model

Save the complete trained preprocessing + model workflow as:

churn_pipeline.pkl

The saved file must be usable directly by the Streamlit app.

13. Reload and Test the Saved Model

Load churn_pipeline.pkl again and make at least ONE manual prediction to prove the saved file works correctly.

Part B — Streamlit Application

Create:

app.py

The Streamlit app must:

- have a clear title and project description
- collect user inputs for all model features
- use suitable widgets such as:
 - number_input
 - selectbox
 - slider
- load churn_pipeline.pkl
- include a Predict Churn button
- display the prediction clearly
- show Yes if churn is predicted
- show No if churn is not predicted
- handle user input without crashing

Bonus

If your selected model supports it, display prediction probability/confidence.

Part C — Local Testing

Run:

streamlit run app.py

Test multiple customer examples and confirm the application works before deployment.

Part D — GitHub

Create a GitHub repository containing at least:

- customer_churn_data.csv
- model_training.ipynb
- churn_pipeline.pkl
- app.py
- requirements.txt
- README.md

Do not upload unnecessary environment folders such as venv.

Part E — Streamlit Community Cloud Deployment

Deploy the app from your GitHub repository.

After deployment, test the public application.

Final Submission

Submit:

1. Jupyter Notebook file
2. Dataset CSV
3. Trained .pkl file
4. app.py
5. requirements.txt
6. GitHub repository link
7. Live Streamlit app link
8. Short project report or README

Important Rules

- Notebook must run from top to bottom without errors.
- Do not manually hard-code predictions.
- The Streamlit app must use the model trained in the notebook.
- Use the same preprocessing at training time and prediction time.
- Explain important decisions in your own words.
- Copied solutions without understanding may be questioned in viva.

Suggested Marking (100)

- Data understanding & quality checks: 10
- Data cleaning: 10
- EDA & observations: 15
- Preprocessing: 15
- Model training: 15
- Model evaluation & comparison: 15
- Saving/reloading model: 5
- Streamlit app: 10
- GitHub + deployment: 5